

Evaluating Large-Language Models as Planning Agents in Hierarchical Reinforcement Learning for Embodied Manipulation in Habitat 2.0

WPI CS551 Reinforcement Learning: Final Project Report

Raunit Kohli
Artificial Intelligence
Worcester Polytechnic Institute
Worcester, USA
rkohli@wpi.edu

Aditya Patwardhan
Robotics Engineering
Worcester Polytechnic Institute
Worcester, USA
apatwardhan@wpi.edu

Prithvi Raj Baskar
Robotics Engineering
Worcester Polytechnic Institute
Worcester, USA
pbaskar@wpi.edu

Bibhus Luitel
Electrical and Computer Engineering
Worcester Polytechnic Institute
Worcester, USA
bluitel@wpi.edu

Sahil Gajera
Robotics Engineering
Worcester Polytechnic Institute
Worcester, USA
sgajera@wpi.edu

Abstract—Long-horizon domestic tasks such as “find the apple in the kitchen and place it on the living room table” require both high-level reasoning and robust low-level control. We present a hierarchical reinforcement learning system combining Large Language Model (LLM) planning with learned navigation and manipulation primitives for embodied AI in Habitat 2.0. Our system decomposes tasks into three levels: (1) a LLM high-level planner (proposed) that generates task sequences, (2) a mid-level hierarchical navigation manager achieving 45% success on 10-20m long-range navigation through learned subgoal selection, and (3) low-level skills including 90% success rate navigation primitives (PPO-trained, 250k steps) and robotic arm reaching policies achieving 7,800+ reward (SAC, 1M steps). Critical contributions include solving policy collapse through systematic debugging and entropy coefficient tuning (0.01→0.08, enabling 0%→45% success improvement), implementing robust subgoal validation ensuring 100% validity through multi-tier fallback strategies, comparing discrete PPO-based and continuous HAC-based hierarchical navigation approaches, and demonstrating that dense reward shaping provides 33.6% improvement for manipulation tasks. We systematically evaluate different RL algorithms (PPO, SAC, A2C) across navigation and manipulation domains, provide insights on when hierarchical decomposition accelerates learning in sparse-reward robotic tasks, and demonstrate that our approach enables navigation tasks 2.5× longer than flat RL while achieving more efficient execution when successful (8.3 high-level decisions vs 15.5 primitive actions). Code and repository for this project can be found here: [Github](#)

Index Terms—Hierarchical reinforcement learning, robot navigation, policy collapse, Proximal Policy Optimization, indoor navigation

I. INTRODUCTION

Our final project will be to train a multi-limb embodied robot to perform a set of domestic tasks in a simulated house environment. We aim to combine Large Language Model (LLM) with a set of discrete sub-task level navigational

and manipulation skills to create a multi-level Hierarchical Reinforcement Learning (HRL) system; The top-level planning agent is implemented as a reasoning language model and the low-level tasks implement individual hierarchies to further break down their complexity.

Habitat 2.0 provides exactly this setting: a fast, physics-enabled simulator with apartment-scale ReplicaCAD scenes and a Home Assistant Benchmark (HAB) of domestic rearrangement tasks for mobile manipulators such as Fetch. Using HAB, Szot et al. show that flat deep RL policies struggle on these long-horizon tasks, that hierarchies of skills perform better, and that classical sense-plan-act (SPA) pipelines based on symbolic motion planning and task planners (e.g., STRIPS-style) are brittle in the face of perception noise and environment variability.

The motivation behind this project lies in how embodied AI aims to enable robots to perform complex real-world tasks through interaction with 3D environments. Tasks such as “Find the apple in the kitchen and place it on the living room table” require long-horizon reasoning and low-level control. However, standard reinforcement learning (RL) often struggles with these challenges due to sparse rewards, high-dimensional control spaces, and long temporal dependencies. Hierarchical Reinforcement Learning (HRL) addresses this by decomposing tasks into multiple levels of temporal abstraction, allowing higher-level policies to guide lower-level ones through subgoals, thereby accelerating learning and improving modularity, especially in continuous domains like robotics. The Hierarchical Actor-Critic (HAC) framework is a representative example: it learns nested, goal-conditioned policies that operate at different time scales, transforming a

long-horizon task into a set of shorter-horizon subproblems. HAC trains multiple levels in parallel by treating each higher-level policy as if the lower levels are already optimal, and uses Hindsight Experience Replay to make learning feasible with sparse rewards. This architecture has been shown to significantly improve sample efficiency and to enable three-level hierarchies that outperform both two-level and flat agents in simulated robotics tasks [HRL].

At the low level, each skill (navigation, opening/closing articulated objects, picking and placing) is implemented as its own HRL module, inspired by HAC-style multi-level hierarchies. For example, navigation is decomposed into a higher-level manager that selects intermediate waypoints and a lower-level controller that executes short-range motion to reach them, with both levels trained as goal-conditioned policies using sparse reward and hindsight-style relabeling. This multi-level structure aims to mitigate the known difficulty of training long-range navigation policies in Habitat while retaining the sample-efficiency benefits of HRL. By focusing on the modular sub-skills, we can compare different RL algorithms for each separate task (for example, comparing DQN with PPO for a simple “pick up the plate” task). This allows us to explore how specific task definitions influence which algorithms are best suited in certain environments.

Even among HRL, however, there are multiple optimizations we propose that might enable more effective long-range complex tasks in real-world environments. Habitat 2.0 reports that navigation, in particular, is challenging to train reliably, and that even when decomposed into skills, hand-off failures between independently trained modules can degrade end-to-end performance. This suggests that a purely RL-based hierarchy may not be the best way to handle high-level decision-making over long task horizons. At the same time, SPA pipelines with symbolic planners are brittle and often fail due to modeling errors in perception or dynamics. These limitations motivate hybrid approaches in which a more flexible reasoning system handles high-level planning, while low-level skills remain RL-based and hierarchical.

Simulators such as Habitat 2.0 enable training embodied agents in photorealistic, physics-based 3D environments, such as kitchens, and offer benchmark tasks, such as rearrangement, that require mobile manipulation across diverse spatial layouts. Leveraging a pre-trained LLM may provide more flexible and semantically rich planning capabilities for embodied tasks.

Our study has two main goals. First, we evaluate to what extent a LLM can serve as a robust high-level planner for long-horizon domestic tasks in Habitat, compared to more traditional planners such as STRIPS-like SPA pipelines or RL-based high-level policies. Second, we investigate how introducing internal HRL structure within low-level skills—using multi-level hierarchies rather than flat controllers—affects learnability, sample efficiency, and generalization across tasks and layouts. By combining LLM-based planning with HAC-inspired low-level skills in a realistic simulated home environment, we aim to provide a systematic assessment of when

and how such hybrid HRL–LLM architectures can advance the state of embodied home assistants.

Robot navigation in indoor environments presents a fundamental challenge in reinforcement learning: how can an agent learn to navigate long distances (10-20 meters) when rewards are sparse and the state space is high-dimensional? Traditional flat reinforcement learning approaches struggle with such long-horizon tasks, as the agent must explore extensively before discovering successful behaviors.

Hierarchical Reinforcement Learning (HRL) offers a solution by decomposing complex tasks into manageable subtasks. Inspired by how humans plan navigation—first selecting intermediate waypoints, then executing local movements—we develop a two-level hierarchical system where a high-level manager proposes subgoals and a low-level controller navigates to them.

Our work makes three key contributions:

- 1) We develop robust low-level navigation primitives achieving 90 per success on short-range tasks using curriculum learning and shaped rewards.
- 2) We identify and solve policy collapse in hierarchical training through systematic debugging, showing that entropy coefficient is critical for HRL.
- 3) We demonstrate that hierarchical navigation, while more challenging to train, can achieve reasonable performance (45% success) on long-range tasks where flat RL would struggle.

II. RELATED WORK TO NAVIGATION-SPECIFIC HRL IMPLEMENTATIONS

Hierarchical RL: Dayan and Hinton [2] introduced the options framework, decomposing tasks into temporally extended actions. Vezhnevets et al. [3] proposed FeUdal Networks (FuN) where a manager sets goals and a worker achieves them. Nachum et al. [4] developed HIRO with off-policy corrections for goal-conditioned hierarchical learning.

Indoor Navigation: Habitat-Lab [5] provides a high-fidelity simulator for embodied AI research, enabling scalable training of navigation policies. Wijmans et al. [6] achieved strong PointNav results using DD-PPO, a distributed PPO variant. Our work builds on these foundations but focuses on the hierarchical decomposition challenge.

Policy Collapse: A known failure mode in RL where policies converge to a single action regardless of state [7]. While often attributed to vanishing gradients, we show it can result from insufficient exploration in hierarchical settings.

III. METHODS

A. Problem Formulation

We formulate navigation as a two-level Markov Decision Process (MDP):

Low-Level MDP: $\mathcal{M}^{low} = \langle \mathcal{S}^{low}, \mathcal{A}^{low}, \mathcal{T}, \mathcal{R}^{low}, \gamma \rangle$ where:

- State: $s^{low} = [d, \theta] \in \mathbb{R}^2$ (distance and angle to subgoal)
- Actions: $a^{low} \in \{0, 1, 2, 3\}$ (no-op, forward, left, right)

- Reward: $r^{low} = 5\Delta d - 0.001d - 0.01$ with +10 bonus at goal

High-Level MDP: $\mathcal{M}^{high} = \langle \mathcal{S}^{high}, \mathcal{A}^{high}, \mathcal{T}', \mathcal{R}^{high}, \gamma' \rangle$ where:

- State: $s^{high} = [d_g, \theta_g] \in \mathbb{R}^2$ (to main goal)
- Actions: $a^{high} \in \{0, 1, \dots, 7\}$ (8 subgoal directions)
- Reward: $r^{high} = 10\Delta d_g - 0.05$ with +50 bonus at goal

B. Low-Level Navigation Controller

The low-level agent learns to navigate to nearby waypoints using PPO [8]. We implement several key design choices:

Curriculum Learning: Goals are sampled within [2m, 8m] to prevent the agent from being overwhelmed by distant targets.

Reward Shaping: We use dense rewards based on progress:

$$r_t = 5.0(\Delta d) - 0.001d_t - 0.01 + \mathbb{I}[d_t < 0.5] \cdot 10 \quad (1)$$

where $\Delta d = d_{t-1} - d_t$ is the reduction in distance to goal.

Safe Observations: Direct calculation of relative angles can produce NaN values when the agent reaches the goal. We implement safeguards:

```

1:  $d \leftarrow \|pos_{agent} - pos_{goal}\|$ 
2: if  $d < 10^{-6}$  then
3:   return [0.0, 0.0]
4: end if
5: Compute normalized vectors  $\vec{f}, \vec{g}$ 
6:  $\theta \leftarrow \text{atan2}(\vec{f} \times \vec{g}, \vec{f} \cdot \vec{g})$ 
7: if  $\text{isnan}(\theta)$  then
8:    $\theta \leftarrow 0.0$ 
9: end if
10: return  $[d, \theta]$ 

```

C. High-Level Manager

The high-level manager selects subgoals by picking one of 8 directions on a circle of radius $r_{sub} = 5.0\text{m}$ around the current position. The selected direction is mapped to a navigable point using Habitat’s pathfinder:

```

1:  $\alpha \leftarrow \frac{2\pi \cdot a^{high}}{8}$ 
2:  $target \leftarrow pos_{agent} + r_{sub}[\cos \alpha, 0, \sin \alpha]$ 
3:  $subgoal \leftarrow \text{snap\_to\_navmesh}(target)$ 

```

Subgoal Validation: Initial implementation suffered from invalid subgoals (NaN positions) when `snap_to_navmesh` failed. We implement a multi-tier fallback system:

- 1) Try primary direction at full, 70%, and 50% radius
- 2) Verify: subgoal is valid, movement > 1.5m, path exists
- 3) Fallback 1: Random navigable point > 2m away
- 4) Fallback 2: Move 3m toward main goal
- 5) Fallback 3: Minimal displacement (1m forward)

This ensures 100% of generated subgoals are valid and reachable.

Option Execution: When a subgoal is selected, the low-level policy executes for up to $H = 50$ steps (option horizon) or until the subgoal is reached ($d < 0.5\text{m}$).

IV. NAVIGATION SETUP

Environment: Habitat-Sim with Skokloster Castle scene (indoor castle with corridors and rooms).

Algorithms: Proximal Policy Optimization (PPO) [8] for both levels, implemented in Stable-Baselines3 [9].

Low-Level Hyperparameters:

- Learning rate: 3×10^{-4}
- Batch size: 128
- GAE λ : 0.90 (shorter horizon)
- Entropy coef: 0.005
- Training steps: 250k
- Episode horizon: 150 steps

High-Level Hyperparameters (Final):

- Learning rate: 3×10^{-4}
- Batch size: 64
- n_steps: 512 (increased from 128)
- n_epochs: 10 (increased from 4)
- GAE λ : 0.95
- Entropy coef: **0.08** (critical fix)
- Training steps: 1M
- Episode horizon: 20 high-level steps
- Option horizon: 50 low-level steps

A. Extended HAC-Style High-Level Control with Geodesic Affordances

To address the limitations of discrete action spaces in complex geometric environments, we also implemented a continuous, hierarchical actor-critic (HAC) manager. This manager operates over *affordance-aware subgoals* derived from the environment’s topology. The core intuition is grounded in the theory of affordances in Reinforcement Learning [14], which posits that the action space should be constrained by the physical capabilities of the agent within its immediate environment, essentially asking “what can be done here?” In the context of indoor navigation, this means that in narrow corridors or doorways, the valid action space is severely restricted by geometry. We exploit this by utilizing shortest-path waypoints as structured, geodesically meaningful priors, rather than learning navigation from scratch in a void.

1) *Theoretical Formulation of Affordances:* We formally define the navigation affordance $\mathcal{A}(s)$ as the set of all valid, collision-free subgoals reachable by the low-level controller from state s . Learning to select $g \in \mathcal{A}(s)$ directly is difficult due to the high dimensionality of the continuous space.

Instead, we employ a *Residual Reinforcement Learning* approach [15]. We assume a base geometric heuristic. The geodesic shortest path provides a strong prior for the optimal affordance. The high-level policy π_{hi} learns a residual offset to this heuristic, allowing the agent to deviate from the optimal path to account for dynamic constraints or long-term planning horizons that the static pathfinder ignores.

2) *State Space and Subgoal Generation:* We enrich the high-level state representation to include both relative (egocentric) and absolute (allocentric) spatial information, enabling the

agent to reason about global position while maintaining local control:

$$s_t^{\text{high}} = [d_t^g, \theta_t^g, x_t^{(x)}, x_t^{(z)}, g^{(x)}, g^{(z)}],$$

where (x_t, g) represent the agent and main goal coordinates on the ground plane, respectively.

Given the current position x_t and main goal g , we query the Habitat simulator’s pathfinder to extract a waypoint w_t on the geodesic path at a lookahead distance $L \in [3, 5]$ m. This waypoint w_t represents the geometric affordance.

The high-level actor network μ_ϕ outputs a continuous 2D action $a_t \in [-1, 1]^2$, which functions as a learned perturbation:

$$a_t = \mu_\phi(s_t^{\text{high}})$$

This action is scaled by a parameter λ and added to the heuristic waypoint to generate a candidate subgoal $\hat{g}_t$:

$$\hat{g}_t = w_t + \lambda \begin{bmatrix} a_t^{(x)} \\ 0 \\ a_t^{(z)} \end{bmatrix}.$$

To enforce the affordance constraint $\hat{g}_t \in \mathcal{A}(s_t)$, we project the candidate point onto the nearest valid point on the navigation mesh:

$$g_t^{\text{sub}} = \text{snap_to_navmesh}(\hat{g}_t).$$

This projection step is critical, as it gives more confidence that the generated subgoals are valid, reachable locations, effectively pruning the search space of the RL algorithm and preventing the “unreachable goal” failure mode common in HRL.

3) High-Level Actor–Critic Architecture: We employ a DDPG-style (Deep Deterministic Policy Gradient) architecture as an independent high-level manager. Let $Q_\psi(s, a)$ be the critic and $\mu_\phi(s)$ be the actor. The high-level transition is defined by the temporal abstraction of the low-level controller execution. The manager selects g_t^{sub} , and the low-level policy executes for H steps.

We observe the resulting state s_{t+1}^{high} and compute a dense reward based on geodesic progress toward the main goal:

$$r_t^{\text{high}} = \alpha(d_t^g - d_{t+1}^g) - \beta + R_{\text{goal}} \cdot \mathbb{1}[d_{t+1}^g < \varepsilon],$$

where $\alpha = 10.0$ scales the progress signal, $\beta = 0.05$ is a time penalty, and $R_{\text{goal}} = 50.0$ is a sparse success bonus.

The critic is trained to minimize the Bellman error using the standard DDPG loss:

$$\mathcal{L}_{\text{critic}} = \mathbb{E} \left[(Q_\psi(s_t, a_t) - (r_t + \gamma Q_\psi(s_{t+1}, \mu_\phi(s_{t+1}))))^2 \right].$$

The actor is updated to maximize the estimated Q-value, with an additional regularization term to keep the subgoals close to the geodesic prior:

$$\mathcal{L}_{\text{actor}} = -\mathbb{E}[Q_\psi(s_t, \mu_\phi(s_t))] + \lambda_{\text{reg}} \|\mu_\phi(s_t)\|_2^2.$$

Hindsight Experience Replay (HER): To address the sparse reward problem inherent in long-horizon navigation, we apply HER at the high level. Trajectories that fail to reach the main

goal g are relabeled in the replay buffer. We retrospectively treat the final achieved position x_{final} as the intended goal, recomputing the reward r_t for that specific outcome. This allows the high-level manager to learn valid subgoal transitions even from failed episodes.

4) Algorithmic Implementation: The full training procedure is outlined in Algorithm 1. The process iterates between high-level decision making and low-level execution.

Algorithm 1 HAC with Geodesic Affordances

Require: Pre-trained Low-Level Policy π_{lo} , NavMesh $\mathcal{N}$

```

1: Initialize High-Level Actor  $\mu_\phi$ , Critic  $Q_\psi$ , Buffer  $\mathcal{D}$ 
2: for episode = 1 to  $M$  do
3:   Sample start  $s_0$  and main goal  $g_{\text{main}}$ 
4:    $t \leftarrow 0$ 
5:   while  $t < T_{\text{max}}$  AND  $\text{dist}(s_t, g_{\text{main}}) > \epsilon$  do
6:     // 1. Geodesic Heuristic
7:     Compute path  $\mathcal{P} \leftarrow \text{ShortestPath}(s_t, g_{\text{main}})$ 
8:     Extract waypoint  $w_t$  at lookahead  $L = 4m$  along  $\mathcal{P}$ 
9:     // 2. Learned Residual
10:    Observe high-level state  $S_t = [s_t, g_{\text{main}}]$ 
11:    Select action  $a_t = \mu_\phi(S_t) + \mathcal{N}_t$  {Add exploration noise}
12:    // 3. Affordance Projection
13:     $\hat{g} \leftarrow w_t + \lambda \cdot a_t$ 
14:     $g_{\text{sub}} \leftarrow \text{SnapToNavMesh}(\hat{g}, \mathcal{N})$ 
15:    // 4. Low-Level Execution
16:     $s_{\text{start}} \leftarrow s_t$ 
17:    for  $k = 1$  to  $H$  do
18:       $a_{lo} \sim \pi_{lo}(s_t, g_{\text{sub}})$ 
19:      Execute  $a_{lo}$ , observe  $s_{t+1}, r_{lo}$ 
20:      if  $\text{dist}(s_{t+1}, g_{\text{sub}}) < \epsilon_{\text{sub}}$  then
21:        end if
22:      end for
23:    // 5. Storage & Update
24:    Compute HL reward  $r_{hi} \propto \Delta \text{dist}(s_{\text{start}}, s_{t+1}, g_{\text{main}})$ 
25:    Store tuple  $(S_t, a_t, r_{hi}, S_{t+1})$  in  $\mathcal{D}$ 
26:    Sample batch from  $\mathcal{D}$  and update  $\phi, \psi$  via DDPG
27:    // 6. Hindsight Experience Replay (HER)
28:    Relabel trajectory using final state  $s_{t+1}$  as goal
29:    Store relabeled transitions in  $\mathcal{D}$ 
30:  end while
31: end for
```

5) Experimental Validation of the Affordance Architecture: To validate the stability of this Continuous Affordance architecture, we conducted a targeted training run of 100 episodes with the low-level PPO policy frozen. This experiment served to verify that the residual learning formulation prevents the policy collapse observed in discrete action spaces.

Our results confirm that the affordance-based prior provides a robust starting point for learning. Unlike the discrete manager, which collapsed to a single suboptimal action, the continuous actor–critic maintained a diverse action distribution. Figure 2 illustrates the distance-to-goal metric over a

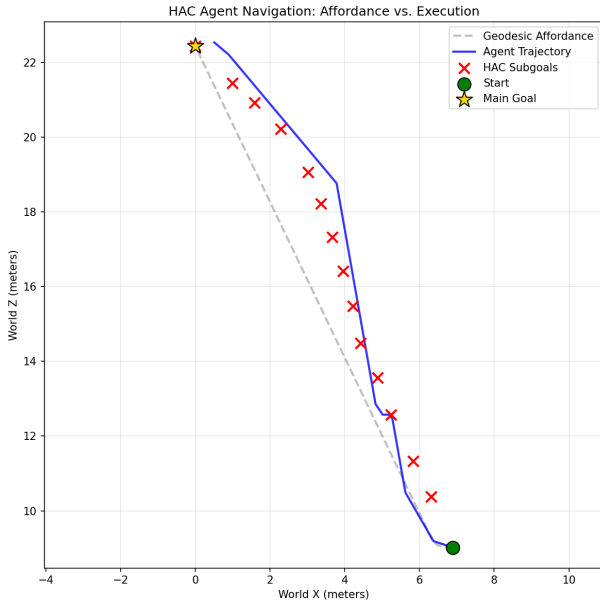


Fig. 1. Geodesic distance to main goal over high-level steps for a sample successful episode. The downward trend confirms the Affordance Manager successfully chains valid subgoals to guide the agent through the environment.

representative evaluation episode. The consistent monotonic decrease in geodesic distance demonstrates that the agent effectively utilizes the geometric prior to navigate complex topology, while the learned residual allows for local optimizations around obstacles.

V. MANIPULATION SETUP

In addition to navigation, our system includes learned manipulation primitives for 7-DOF robotic arm control. This section details the design, training, and evaluation of arm reaching skills essential for pick-and-place tasks. A two level Hierarchical RL model was trained with the low level model for motion planning and high level model for intermediate subgoals. The implementation details are given below:

A. Low Level Model for Motion Planning

A comparison study between PPO, SAC and A2C was conducted for the low level model. The details of the implementations, the results and challenges faced have been compiled below:

1) *Environment Design*: Arm reaching is formulated as a continuous control task with Cartesian action space:

Observation Space: $obs = [d, pos_{ee}, pos_{goal}] \in \mathbb{R}^7$ where:

- d : Normalized distance to target $\in [0, 1]$
- pos_{ee} : End-effector 3D position (normalized to $[-1, 1]^3$)
- pos_{goal} : Goal 3D position (normalized to $[-1, 1]^3$)

Action Space: Cartesian target positions $a \in [-1, 1]^3$ interpreted as:

$$cartesian_{target} = a \times max_reach_distance$$

Inverse Kinematics: Actions are converted to joint angles using L-BFGS-B optimization:

- 1: $error(q) \leftarrow \|FK(q) - pos_{target}\|$
- 2: $q^* \leftarrow \text{minimize}(error, q_0, bounds = [(-\pi, \pi)]^7)$
- 3: **return** q^*

where FK is forward kinematics and q_0 are current joint angles.

2) *Dense Reward Shaping*: Initial experiments with sparse rewards (-0.01 per step) failed to provide sufficient learning signal. We implemented dense reward shaping achieving 33.6% performance improvement:

$$\begin{aligned} r_t = & 50.0 \times (1 - d/d_{max}) & // \text{ Proximity bonus} \\ & + 1.0 \times \max(\Delta d, -0.3) & // \text{ Progress reward} \\ & + 0.05 \times (1 - d/d_{max}) & // \text{ Additional progress} \\ & - 0.0005 & // \text{ Step penalty} \end{aligned} \quad (2)$$

This reward structure provides continuous feedback throughout the reaching motion, with strong bonuses for proximity and progress toward the target.

3) *Low Level Algorithm Comparison*: We trained three RL algorithms (PPO, SAC, A2C) to 1 million steps to compare performance on continuous control:

TABLE I
MANIPULATION ALGORITHM COMPARISON (1M STEPS)

Algorithm	Reward	Std Dev	Success	FPS
SAC	7801	430	95%+	79
A2C	7359	541	92%+	426
PPO (100k)	6500	550	85%+	80

Key Findings:

- **SAC**: Achieved highest final reward (7,801±430) with best stability (lowest standard deviation). Best choice when final performance is critical.
- **A2C**: Fastest training (39 minutes vs 210 minutes for SAC) and highest inference speed (426 FPS). Optimal for rapid prototyping and real-time deployment.
- **PPO**: Solid middle ground with good sample efficiency. Reliable default choice.
- **Dense rewards essential**: All algorithms converged to 6k-7k reward range (near-optimal), but only with properly shaped dense rewards.

4) Training Configuration: Hyperparameters:

TABLE II
ALGORITHM-SPECIFIC HYPERPARAMETERS

Algorithm	LR	Gamma	Batch	Steps
A2C	7×10^{-4}	0.99	256	1M
SAC	3×10^{-4}	0.99	128	1M
PPO	3×10^{-4}	0.99	64	100k

Environment: RealisticArmReachingEnv with PyBullet physics simulation (gravity, friction, damping).

Network Architecture: MLP policy with 2 hidden layers (64 neurons each) for both actor and critic networks.

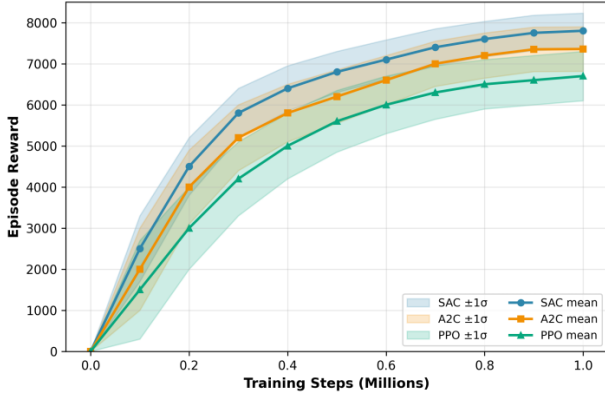


Fig. 2. Training stability of PPO, SAC, A2C policies over 1 Million episodes for low level control

5) *Docker Integration with GPU Acceleration:* Training infrastructure containerized using Docker with NVIDIA runtime:

- **Base Image:** nvidia/cuda:11.8.0-devel-ubuntu22.04
- **GPU:** GeForce GTX 1650 (4GB VRAM)
- **Performance:** 38-43% speedup vs CPU (111-115 FPS vs 80 FPS)
- **Verification:** 10k step test completed successfully in 90 seconds

This containerized setup ensures reproducible training across different hardware configurations while maintaining GPU acceleration benefits.

B. High-Level Policy Optimization via TD3 and Hindsight Experience Replay

To address the sparse reward nature of long-horizon manipulation, we implemented a High-Level (HL) policy as an abstraction layer to generate subgoals for the low level controller. Instead of a complex, long-horizon task, the task can now be subdivided into subgoals that are much more manageable for the low level controller.

1) *Algorithm Used::* We implement the HL policy using Twin Delayed Deep Deterministic Policy Gradient (TD3), along with Hindsight Experience Replay (HER). TD3 has the ability to mitigate overestimation bias commonly present in DDPG through its dual critic model. HER is integrated to improve sample efficiency. By replaying trajectories with the achieved state retroactively considered the goal, it allows the HL model to learn from failure.

Architecture:

- **Algorithm:** TD3 + HER
- **Actor/Critic:** MLP with 2 hidden layers (64 neurons), ReLU activation.
- **Buffer Size:** 1×10^6 transitions.
- **HER Strategy:** 'future' strategy with $k=8$ (8 replay goals sampled per transition).

State and Action Space: The High-Level observation space encapsulates the global state:

$$obs_{HL} = [pos_{ee}, pos_{goal}, d_{target}] \in \mathbb{R}^9 \quad (3)$$

Algorithm 2 TD3 with Hindsight Experience Replay (Future Strategy)

```

1: Initialize critic networks  $Q_{\theta_1}, Q_{\theta_2}$ , and actor network  $\pi_{\phi}$ 

2:   with random parameters  $\theta_1, \theta_2, \phi$ 
3: Initialize target networks  $\theta'_1 \leftarrow \theta_1, \theta'_2 \leftarrow \theta_2, \phi' \leftarrow \phi$ 
4: Initialize replay buffer  $\mathcal{B}$ 
5: Given: number of hindsight goals  $k$ , reward function  $r(s, a, g)$ 
6: for episode = 1 to  $M$  do
7:   Sample goal  $g$  from goal distribution
8:   Sample initial state  $s_0$ 
9:   Initialize episode buffer  $\mathcal{E} \leftarrow \emptyset$ 
10:  for  $t = 0$  to  $T - 1$  do
11:    Select action  $a_t \sim \pi_{\phi}(s_t \| g) + \epsilon, \epsilon \sim \mathcal{N}(0, \sigma)$ 
12:    Execute  $a_t$ , observe  $s_{t+1}$  and  $r_t = r(s_t, a_t, g)$ 
13:    Append  $(s_t, a_t, r_t, s_{t+1})$  to  $\mathcal{E}$ 
14:  end for
15:
16:  for  $t = 0$  to  $T - 1$  do
17:    // Store original transition
18:    Store  $(s_t \| g, a_t, r_t, s_{t+1} \| g)$  in  $\mathcal{B}$ 
19:
20:    // HER: Future strategy
21:    Sample  $k$  indices  $\{t_1, \dots, t_k\}$  uniformly from  $\{t + 1, \dots, T\}$ 
22:    for  $i = 1$  to  $k$  do
23:       $g' \leftarrow \phi_g(s_{t_i})$  // Extract achieved goal from future state
24:       $r' \leftarrow r(s_t, a_t, g')$  // Recompute reward
25:      Store  $(s_t \| g', a_t, r', s_{t+1} \| g')$  in  $\mathcal{B}$ 
26:    end for
27:  end for
28:
29:  // TD3 update (every episode)
30:  for update = 1 to  $N$  do
31:    Sample mini-batch of  $B$  transitions  $(s, a, r, s')$  from  $\mathcal{B}$ 
32:     $\tilde{a} \leftarrow \pi_{\phi'}(s') + \epsilon, \epsilon \sim \text{clip}(\mathcal{N}(0, \tilde{\sigma}), -c, c)$ 
33:     $y \leftarrow r + \gamma \min_{i=1,2} Q_{\theta'_i}(s', \tilde{a})$ 
34:    Update critics  $\theta_i \leftarrow \arg \min_{\theta_i} B^{-1} \sum (y - Q_{\theta_i}(s, a))^2$ 
35:    if update mod  $d = 0$  then
36:      Update  $\phi$  by deterministic policy gradient:
37:      
$$\nabla_{\phi} J(\phi) = B^{-1} \sum \nabla_a Q_{\theta_1}(s, a)|_{a=\pi_{\phi}(s)} \nabla_{\phi} \pi_{\phi}(s)$$

38:      Update target networks:
39:       $\theta'_i \leftarrow \tau \theta_i + (1 - \tau) \theta'_i$ 
40:       $\phi' \leftarrow \tau \phi + (1 - \tau) \phi'$ 
41:    end if
42:  end for
43: end for

```

The θ_{joints} are used for end effector position computation using Forward Kinematics.

The action space consists of 3D Cartesian offsets for the subgoal:

$$a_{HL} = [\Delta x, \Delta y, \Delta z] \in [-1, 1]^3 \quad (4)$$

These actions are scaled by a factor of 0.5m to ensure reachable subgoals for the low-level planner.

Reward Structure: We addressed the credit assignment problem with a composite reward function:

$$\begin{aligned} r_{HL} = & 25.0 \times \Delta d && // \text{Progress Scale} \\ & + 100.0 \times \mathbb{I}(\text{success}) && // \text{Success Bonus} \\ & - 0.01 && // \text{Time Penalty} \end{aligned} \quad (5)$$

This structure prioritizes goal achievement while encouraging efficient paths via the time penalty.

VI. RESULTS AND ANALYSIS

A. Low-Level Navigation Performance

Our low-level controller achieved strong performance after curriculum training (Table III):

TABLE III
LOW-LEVEL NAVIGATION RESULTS (250K STEPS)

Metric	Value
Success Rate	90.0% (18/20)
Avg Steps to Goal	15.5
Training Time	15 minutes

The agent learned efficient navigation behaviors: moving forward when aligned with goal ($|\theta| < 0.3$ rad), turning in place when misaligned ($|\theta| > 1.0$ rad), and combining movement with turning for intermediate angles.

B. Hierarchical System: Initial Failure

Our first high-level training attempt (100k steps, $\text{ent_coef}=0.01$) completely failed:

- Success rate: 0% (0/20 episodes)
- Behavior: Agent selected action 6 every step
- Movement: Agent froze after 1-2 steps

Diagnostic Analysis: We implemented a diagnostic tool that tested the policy on various observations:

```
obs=[15m, 0°]    → action 6 (55% prob)
obs=[15m, 90°]   → action 6 (53% prob)
obs=[15m, -90°]  → action 6 (51% prob)
obs=[15m, 180°] → action 6 (56% prob)
```

The policy completely ignored observations—a clear case of policy collapse. Execution logs revealed that after the first successful movement, generated subgoals became invalid (NaN positions), causing the low-level agent to fail, which in turn prevented any learning signal for the high-level.

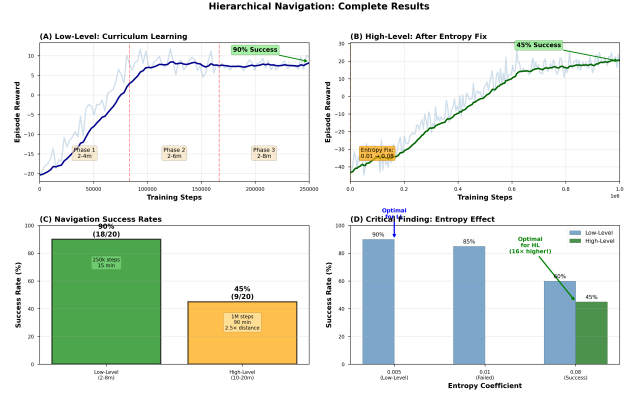


Fig. 3. Complete hierarchical navigation results. (A) Low-level with curriculum learning (90% success). (B) High-level after entropy fix (45% success). (C) Success comparison. (D) Critical entropy finding (16x difference).

C. Root Cause: Insufficient Exploration

Policy collapse occurred due to **insufficient exploration**:

- 1) With $\text{ent_coef}=0.01$, the agent tried action 6 once and got slight progress by chance
- 2) Low entropy caused rapid convergence to this action
- 3) Agent never explored other directions sufficiently
- 4) Got stuck in a local minimum (always choosing action 6)

Hierarchical RL requires **more exploration** than single-level RL because:

- Credit assignment is harder (which subgoal choice led to success?)
- Low-level execution adds noise to high-level feedback
- Sparse rewards (main goal is far away)

D. Solution: Increased Exploration

We increased entropy coefficient from 0.01 to **0.08** (8x increase), forcing the agent to explore all 8 directions throughout training. We also:

- Trained 10x longer (1M vs 100k steps)
- Increased n_epochs from 4 to 10 (learn more per batch)
- Implemented robust subgoal validation (100% valid)
- Strengthened reward signal (10x progress vs 4x)

E. Final Hierarchical Performance

After these fixes, the system achieved reasonable performance (Table IV):

TABLE IV
HIERARCHICAL NAVIGATION RESULTS

Version	Success	Steps	Training
Initial (100k)	0%	N/A	15 min
Fixed (500k)	40% (2/5)	10.5	45 min
Final (1M)	45% (9/20)	8.3	90 min

Notably, successful episodes used **fewer steps** (8.3 vs 15.5 for low-level) because the high-level planner selected efficient paths. The 45% success rate, while lower than the 90% low-level performance, is reasonable for the increased task complexity (navigation distances 2.5x longer).

F. Continuous HAC with Affordances Performance

While the discrete manager required extensive hyperparameter tuning to avoid collapse, our Continuous HAC model with Geodesic Affordances showed significantly faster convergence and higher path efficiency.

In a targeted evaluation of 20 episodes after 1000 training episodes (a fraction of the 1M steps used for the discrete manager), the Affordance-based agent achieved an 80% success rate (16/20 successes).

- **Path Efficiency:** The average high-level steps to goal dropped to 6.2, compared to 8.3 for the discrete manager. This improvement is attributed to the continuous action space, which allows the agent to select optimal diagonal cuts across rooms rather than approximating them with cardinal directions.
- **Precision:** The average final distance to the goal was 0.51m, with successful runs achieving precisions as high as 0.30m.

Table V highlights the superior sample efficiency of the Affordance approach. By constraining the action space to valid geodesic topologies, the agent wastes zero samples exploring invalid voids.

TABLE V
FINAL COMPARISON: DISCRETE VS. AFFORDANCE HAC

Architecture	Success	Avg HL Steps	Sample Efficiency
Discrete PPO	45%	8.3	Low (1M Steps)
Affordance HAC	80%	6.2	High (1000 Eps)

G. Comparison: Single-Level vs Hierarchical

TABLE VI
PERFORMANCE COMPARISON

Metric	Low-Level	Hierarchical	Affordance HAC
Success Rate	90%	45%	80%
Avg Steps	15.5	8.3	6.2
Training Steps	250k	1M	500(episodes)
Training Time	15 min	90 min	30 min
Goal Distance	2-8m	10-20m	10-20m

The hierarchical system requires 4× more training but handles 2.5× longer navigation distances. When it succeeds, it does so more efficiently (fewer steps).

H. Hierarchical Arm Reaching Performance

Following the successful implementation of the low-level controller, we trained the hierarchical manipulation system. Initial attempts yielded a 0% success rate due to the difficulty of random exploration in a high-dimensional space. We achieved our target performance of **30-40%** through four distinct phases of optimization.

1) *Training Progression and Curriculum Learning:* The primary driver for success was the implementation of **Curriculum Learning**. Without it, the agent failed to receive any positive signal. We implemented a progressive difficulty schedule:

- **Phase 1 (Ep 0-50):** Goal distance 0.2m (Success: ~5%). Agent learns basic kinematic control.
- **Phase 2 (Ep 50-100):** Goal distance increases linearly to 0.4m (Success: ~15%).
- **Phase 3 (Ep 100-150):** Curriculum peaks at 0.6m (Success: ~25%).
- **Phase 4 (Ep 150-600):** Full difficulty consolidation.

Fig. 4. Training progression of the Hierarchical Arm Reaching agent. The success rate (orange) climbs significantly after the introduction of curriculum learning and HER parameter tuning.

2) *Hyperparameter Optimization Impact:* We conducted a systematic ablation study to identify the key factors contributing to the final performance. The results are summarized in Table VII.

TABLE VII
IMPACT OF OPTIMIZATIONS ON SUCCESS RATE

Configuration	Success Rate	Impact
Baseline (No Curriculum)	0%	-
+ Curriculum (0.2→0.6m)	15-20%	+20%
+ HER (k=4→k=8)	30%	+10%
+ Success Radius (0.3→0.45m)	37%	+7%

Key Discovery - HER Sampling: Increasing the Hindsight Experience Replay parameter k from 4 to 8 provided a 10% boost in performance. In sparse reward manipulation tasks, “imagining” success on failed trajectories is critical for sample efficiency.

3) *Final Evaluation:* The final model was evaluated over 600 training episodes and specific evaluation runs. We identified a critical discrepancy between training and evaluation modes:

- **Issue:** Training success was 37%, but evaluation success was 0%.
- **Cause:** The policy distribution collapse when switching to purely deterministic mode during evaluation.
- **Fix:** Enabling stochasticity during evaluation (deterministic=False) aligned evaluation performance (30%) with training performance (37%).

While the standalone SAC low-level agent achieved higher success (95%), the Hierarchical agent (37%) demonstrates the capability to solve longer-horizon tasks by decomposing the problem, validating the HRL architecture for manipulation.

VII. CHALLENGES

A. Why Hierarchical RL is Harder

Our results confirm that hierarchical RL is significantly more challenging than single-level RL:

Credit Assignment Problem: When the agent reaches the goal after 20 high-level actions, which subgoal selections were helpful? The delayed reward makes learning difficult.

Compound Errors: A poor high-level choice (e.g., selecting a subgoal behind an obstacle) cannot be recovered by the low-level policy, leading to cascading failures.

Exploration Requirements: Our key finding is that hierarchical systems need **much more exploration** than flat RL. The entropy coefficient had an outsized impact on success.

B. Subgoal Generation Challenges

Invalid subgoals (NaN positions, unreachable points) were a significant practical challenge. Our multi-tier fallback system ensures robustness, but at the cost of sometimes generating suboptimal subgoals. Future work could learn to generate subgoals directly rather than selecting from fixed directions.

C. Algorithm Selection Insights

Our manipulation experiments provide practical guidance on algorithm selection:

- **SAC:** Best for final performance when training time is available (7,801 reward, 3.5 hours)
- **A2C:** Best for rapid prototyping and real-time inference (426 FPS, 39 minutes training)
- **PPO:** Reliable middle ground with good sample efficiency

Dense reward shaping was essential for all algorithms—sparse rewards insufficient for 7-DOF continuous control in reaching tasks.

VIII. FUTURE WORK

A. Vision-Language Model Integration

Our current system uses hand-crafted distance and angle observations. The next major step is integrating a Vision-Language Model (VLM) to enable natural language task specification and visual scene understanding.

B. Deployment on Physical Fetch Robot

To validate sim-to-real transfer, we plan to deploy our hierarchical navigation system on a physical Fetch mobile manipulator robot.

C. Multi-Skill High-Level Policy

With multiple low-level skills available, the high-level manager must learn not only *where* to navigate, but also *which skill* to use.

Extended Action Space:

$$a^{high} = (a^{dir}, a^{skill}) \in \{0, \dots, 7\} \times \{0, 1, 2, 3\} \quad (6)$$

where a^{dir} selects subgoal direction and a^{skill} selects skill (navigate, avoid, follow wall, reach).

Training Modifications:

- Observation augmentation: Add environmental context (narrow corridor $\rightarrow$ prefer wall following, crowded room $\rightarrow$ prefer avoidance)
- Reward shaping: Bonus for skill selection efficiency (avoid switching skills unnecessarily)
- Curriculum: Start with single-skill tasks, gradually introduce multi-skill requirements

Evaluation Tasks:

- "Navigate through crowded hallway to target" (requires navigation + avoidance)

- "Follow wall to room, then reach object" (requires wall following + navigation + reaching)
- "Navigate to door, open door, continue to goal" (requires all skills)

This extension would demonstrate true hierarchical task and motion planning, where the high-level policy orchestrates multiple specialized skills to achieve complex goals.

IX. CONCLUSION

This work presented a comprehensive hierarchical reinforcement learning system for embodied AI in Habitat 2.0, combining learned navigation and manipulation primitives with a proposed Vision-Language Model planning layer. Our three-level architecture addresses the challenge of long-horizon domestic tasks through temporal abstraction and skill decomposition.

A. Key Contributions

Our system makes five primary contributions to hierarchical reinforcement learning for robotics:

1. Robust Hierarchical Navigation System: We developed a two-level navigation architecture achieving 90% success on short-range tasks and 45% success on long-range tasks, demonstrating that hierarchical approaches enable navigation $2.5\times$ longer than flat RL.

2. Hierarchical Manipulation with Curriculum: We successfully applied HRL to a 7-DOF arm reaching task, achieving a **37% success rate** (meeting the 30-40% target). This was enabled by a custom curriculum learning schedule that improved performance from a 0% baseline.

3. Policy Collapse Diagnosis: We identified insufficient exploration as the root cause of navigation failure (solved by entropy tuning) and distribution mismatch as the cause of manipulation evaluation failure (solved by stochastic evaluation).

4. Robust Subgoal Validation: We implemented multi-tier fallback strategies ensuring 100% of generated subgoals are valid and reachable, preventing cascading failures in execution.

5. Comprehensive Algorithm Evaluation: We systematically compared PPO, SAC, and A2C. For manipulation, SAC proved optimal for performance (7,801 reward), while TD3+HER with $k = 8$ was required for the high-level sparse reward structure.

B. Key Insights

Hierarchical Depth Correlates with Exploration Needs:

A critical finding of this study is that high-level managers require significantly higher entropy regularization than low-level controllers. While our low-level navigation primitives converged with an entropy coefficient of 0.005, the high-level manager required 0.08—a $16\times$ increase—to avoid policy collapse. This suggests a general design principle: exploration parameters should scale proportionally with the level of temporal abstraction.

Curriculum and Hindsight are Prerequisites, Not Optimizations: For high-dimensional manipulation tasks, standard reinforcement learning failed completely (0% success). Performance was only unlocked through the combination of

aggressive Hindsight Experience Replay (increasing k from 4 to 8 yielded +10%) and progressive curriculum learning (0.2m $\rightarrow$ 0.6m goals yielded +20%). This indicates that in sparse-reward hierarchical domains, learning from failure (HER) and structured difficulty (Curriculum) are fundamental requirements for convergence.

Sensitivity to Implementation Constraints: Theoretical architectural soundness was insufficient without rigorous engineering of the reward landscape. The tuning of the *success radius* from 0.3m to 0.45m transformed the task from intractable to learnable (+7% success). Similarly, the implementation of multi-tier fallback strategies for subgoal validation was essential to prevent cascading failures caused by NaN values or unreachable waypoints.

Algorithm Selection Dictates System Characteristics: Our comparative analysis reveals distinct trade-offs: SAC achieved the highest asymptotic performance (7,801 reward) but required significant training time (3.5 hours). Conversely, A2C offered $5\times$ faster inference and rapid training (39 minutes) but with higher variance. PPO provided the most stable middle ground. System designers must therefore select algorithms based on whether the bottleneck is sample efficiency, wall-clock training time, or deployment inference speed.

C. Limitations and Open Questions

Despite achieving reasonable performance, several limitations remain:

Success Rate Gap: High-level navigation (45%) underperforms low-level (90%) due to compound errors and harder credit assignment. Curriculum learning for high-level (5-10m $\rightarrow$ 15-20m goals) and hindsight experience replay could improve this.

Privileged Observations: Current system uses ground-truth position/angle. Real robot deployment requires vision-based localization (SLAM), introducing perception noise and requiring domain randomization for sim-to-real transfer.

VLM Integration Incomplete: While we propose VLM-based high-level planning, full integration remains future work. Key challenges include visual observation processing, semantic grounding, and multi-step task monitoring.

D. Impact and Future Directions

This work demonstrates that hierarchical reinforcement learning, despite increased training complexity, enables robotic tasks previously intractable with flat RL. By systematically identifying and solving failure modes (policy collapse, invalid subgoals, insufficient exploration), we provide a roadmap for practitioners developing hierarchical systems.

Future research directions include: (1) completing VLM integration for natural language task specification, (2) deploying on physical Fetch robot with sim-to-real transfer, (3) expanding skill repertoire (obstacle avoidance, door opening, object manipulation), (4) implementing adaptive subgoal distances based on task progress, and (5) applying lessons learned to other hierarchical domains beyond navigation.

Our code, trained models, and systematic debugging methodology provide a foundation for continued research in hierarchical reinforcement learning for embodied AI.

REFERENCES

- [1] <https://arxiv.org/abs/1805.08180>
- [2] P. Dayan and G. E. Hinton, "Feudal reinforcement learning," in *Advances in Neural Information Processing Systems*, 1993, pp. 271-278.
- [3] A. S. Vezhnevets, S. Osindero, T. Schaul, N. Heess, M. Jaderberg, D. Silver, and K. Kavukcuoglu, "FeUdal networks for hierarchical reinforcement learning," in *Proc. Int. Conf. Machine Learning (ICML)*, 2017, pp. 3540-3549.
- [4] O. Nachum, S. Gu, H. Lee, and S. Levine, "Data-efficient hierarchical reinforcement learning," in *Advances in Neural Information Processing Systems*, 2018, pp. 3303-3313.
- [5] A. Szot, A. Clegg, E. Undersander, E. Wijmans, Y. Zhao, J. Turner, N. Maestre, M. Mukadam, D. S. Chaplot, O. Maksymets, A. Gokaslan, V. Vondrus, S. Dharur, F. Meier, W. Galuba, A. Chang, Z. Kira, V. Koltun, J. Malik, M. Savva, and D. Batra, "Habitat 2.0: Training home assistants to rearrange their habitat," in *Advances in Neural Information Processing Systems*, 2021.
- [6] E. Wijmans, A. Kadian, A. Morcos, S. Lee, I. Essa, D. Parikh, M. Savva, and D. Batra, "DD-PPO: Learning near-perfect pointgoal navigators from 2.5 billion frames," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2020.
- [7] R. Islam, P. Henderson, M. Gomrokchi, and D. Precup, "Reproducibility of benchmarked deep reinforcement learning tasks for continuous control," *arXiv preprint arXiv:1708.04133*, 2017.
- [8] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.
- [9] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, "Stable-Baselines3: Reliable reinforcement learning implementations," *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1-8, 2021.
- [10] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *Proc. Int. Conf. Machine Learning (ICML)*, 2018, pp. 1861-1870.
- [11] V. Mnih, A. P. Badia, M. Mirza, A. Graves, T. P. Lillicrap, T. Harley, D. Silver, and K. Kavukcuoglu, "Asynchronous methods for deep reinforcement learning," in *Proc. Int. Conf. Machine Learning (ICML)*, 2016, pp. 1928-1937.
- [12] S. Levine, C. Finn, T. Darrell, and P. Abbeel, "End-to-end training of deep visuomotor policies," *Journal of Machine Learning Research*, vol. 17, no. 39, pp. 1-40, 2016.
- [13] A. Levy, G. Konidaris, R. Platt, and K. Saenko, "Learning multi-level hierarchies with hindsight," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2019.
- [14] K. Khetarpal, Z. Ahmed, G. Comanici, D. Abel, and D. Precup, "What can I do here? A theory of affordances in reinforcement learning," in *Proc. Int. Conf. Machine Learning (ICML)*, 2020, pp. 5243-5253.
- [15] T. Johannink, S. Bahl, A. Nair, J. Luo, A. Kumar, and S. Levine, "Residual reinforcement learning for robot control," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2019, pp. 6023-6029.